

Military Asset Management System

Transparent tracking of military assets across multiple bases

Stack React 18 (Vite) · Node.js · Express · SQLite · JWT

Features Opening / Closing Balance · Net Movement · Purchases · Transfers · Assignments · Expenditures · RBAC · Audit Logging

Quality

77 / 77 API tests pass

39 / 39 browser tests pass

1. Project Overview

1.1 Description

The Military Asset Management System (MAMS) is a web application that lets authorised staff record and monitor the complete lifecycle of military assets — weapons, vehicles, ammunition and equipment — as they move through and out of a chain of bases. The system replaces paper registers and ad-hoc spreadsheets, which are slow to update, easy to lose and impossible to audit, with a single authoritative database and a live dashboard.

Every quantity shown in the interface is derived from a single append-only stock ledger. Purchases, inter-base transfers, asset assignments and write-offs all post signed ledger entries, and the closing balance of any base is simply the net sum of those entries. This makes the headline figures impossible to disagree with the underlying transaction records, which is the property that makes the system trustworthy for accountability purposes.

The application serves three clearly separated roles:

- **Administrator** — full visibility of every base, user management and the audit log.
- **Base Commander** — full transactional control over the base he/she commands.
- **Logistics Officer** — purchasing and transfers only, within their assigned base.

1.2 Assumptions

The following assumptions were made during development. They shape the design and would be revisited with a real user base.

1. **Base-scoped roles.** A user is either global (Administrator) or tied to exactly one base. Most real forces also have regional command tiers; this build models the single-base tier only.
2. **Transfers take time.** Stock dispatched from one base is not available at the destination until the transfer is marked *received*. In-transit stock is therefore visible but not counted as a destination balance.
3. **Assets are homogeneous per type.** Any quantity of "5.56mm ammunition" is fungible with any other, so lots, serials and condition grades are not individually tracked. Serialised high-value items (a single rifle) would need a unit-identity table.
4. **Expenditure is reported against an assignment.** Ammunition issued to a platoon and then fired is recorded against that assignment. A write-off with no assignment (spoilage, loss) is a *direct expenditure*.
5. **Opening balances are supplied per base and equipment type per period.** The system consumes them; it does not calculate them historically.
6. **Stock documents are immutable.** Purchases and transfers are never deleted — they are corrected by counter-transactions, so the audit trail stays intact.
7. **Financial values are informational.** Purchase values are recorded and totalled, but no currency conversion or accounting journal is modelled.
8. **Single language (English), single time basis (UTC).** Dates are stored as ISO-8601 text.

1.3 Limitations

This is a deliberately simple, self-contained build. The following are known and intentional limits, each with a clear path to production.

Limitation	Effect	How it would be fixed
SQLite is a local file	Fine for one server. On a free ephemeral host (e.g. Render's free tier) the disk is wiped on a cold start, so demo data is re-seeded.	Attach a persistent disk, or port the schema to PostgreSQL. Only <code>db.js</code> and the SQL in the routes would change.
JWT is stateless	There is no server-side session, so a deactivated user keeps their token until it expires (8h).	Add a token-version or session table checked on each request, or shorten the expiry.
No login rate-limiting	Password guessing is not throttled at the application layer.	Add a per-IP token bucket (e.g. <code>express-rate-limit</code>) in front of <code>/auth/login</code> .
No CSRF token	Safe as built, because the API is stateless and authenticates with a bearer header, not a cookie.	If cookies are introduced, add CSRF protection at the same time.
No document attachments	Purchase invoices and transfer manifests cannot be uploaded.	Add an object-storage upload endpoint and store the key on the purchase/transfer row.
No notifications	A commander is not emailed when a transfer is dispatched or overdue.	Add a background worker and an email/SMS provider.
No offline / mobile app	Field use requires connectivity; the UI is responsive but browser-based.	Wrap the frontend in a PWA or Capacitor shell with a local queue.

2. Technology Stack & Architecture

2.1 Technology stack

Layer	Technology	Reason for the choice
Frontend	React 18 + Vite 5	Fast dev server, small production bundles, component model suits a forms/dashboard application.
Routing	react-router-dom 6	Client-side routes with protected-route guards and per-role page visibility.
Styling	Hand-written CSS	No framework dependency; a small custom design system keeps the bundle tiny and the look consistent.
Backend	Node.js + Express 4	Minimal, widely understood HTTP layer; the route handlers map 1:1 to the API reference.
Database	SQLite via better-sqlite3	Zero-infrastructure, single file, synchronous API that keeps the ledger writes atomic. Ideal for this scale.
Auth	jsonwebtoken + bcryptjs	Stateless JWT for the API; bcrypt for password hashing (bcryptjs avoids native build steps).
Validation	Hand-written helpers	Explicit, readable validation with clear error messages rather than a schema library.
Logging	Custom audit middleware + <code>audit_logs</code> table	Every transaction is recorded in the database, queryable from the UI — not just written to a log file.
Tests	Node test script + Chrome DevTools Protocol driver	No test framework needed; 77 API assertions and 39 real-browser assertions, all runnable in one command.

2.2 Architecture

A classic three-tier, single-page-application architecture. The browser loads one static bundle; the React client calls a JSON REST API; the API talks to a single SQLite file. There is no separate ORM or service layer — business rules live in the route modules, which keeps the codebase small enough to read end-to-end.

Browser (React 18 SPA)

```
Login → Dashboard → Purchases / Transfers  
Assignments & Expenditures / Users / Audit  
api.js — attaches Bearer token, JSON I/O
```

HTTPS (JSON over /api)

Express server.js

```
authenticate → verify JWT  
requireRole → admin / commander /  
                  logistics  
assertBaseAllowed → own-base scoping  
auditMiddleware → log every action  
  
routes/ : auth · meta · dashboard ·  
          purchases · transfers ·  
          assignments · expenditures · admin  
db.js : schema apply · ledger post
```

```
SQLite military_assets.db (WAL mode)  
stock_ledger ← single source of truth  
audit_logs ← API activity trail
```

2.3 The append-only stock ledger

The most important design decision in the project is that stock is not stored as a mutable running balance on each base/equipment row. Instead, every event writes a signed row into `stock_ledger`, and balances are computed as the sum of those rows. Nothing can drift out of sync because there is nothing to keep in sync.

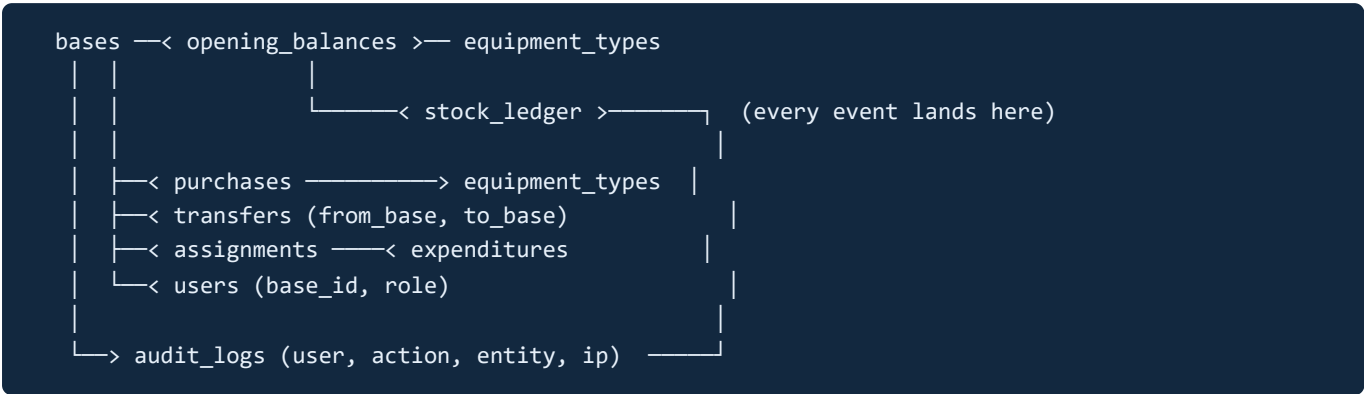
tx_type	direction	Meaning
OPENING	+1	Opening balance for a base/equipment/period
PURCHASE	+1	Stock received from a supplier
TRANSFER_IN	+1	Stock arrived from another base (on receive, or on cancel-back)
TRANSFER_OUT	-1	Stock dispatched to another base
ASSIGNMENT	-1	Stock issued to personnel (leaves base stock)
EXPENDITURE	-1	Direct write-off (spoilage, loss) — <i>not</i> used for expenditure already covered by an assignment

A subtle point worth stating explicitly

When ammunition is issued to a platoon and then fired, the physical stock left the base *once*, at the moment of assignment. Recording a second ledger line at the moment of expenditure would double-count the loss. Therefore an expenditure that is linked to an assignment updates that assignment's `expended_quantity` but writes **no** ledger line. Only a direct write-off, with no assignment, posts a negative `EXPENDITURE` entry. Returning an assignment restores exactly `quantity - expended_quantity`.

3. Data Model / Database Schema

3.1 Entity relationship overview



3.2 Table reference

Table	Purpose and key columns	Notes
bases	id, code, name, location, is_active	Reference data; seeded with 4 bases.
equipment_types	id, code, name, category, unit, is_active	12 seeded types across Weapons, Vehicles, Ammunition, Equipment.
users	id, email, full_name, role, base_id, password_hash, is_active, last_login_at	role ∈ admin commander logistics. Hashes are never returned by the API.
opening_balances	base_id, equipment_type_id, period_start, quantity, created_by	One row per base/equipment/period; unique constraint prevents duplicates.
purchases	reference_no, base_id, equipment_type_id, quantity, unit_cost, supplier, purchased_on, remarks, created_by	Immutable; unique reference_no ; quantity > 0 enforced by CHECK.
transfers	reference_no, from_base_id, to_base_id, equipment_type_id, quantity, status, dispatched_at, received_at, remarks, created_by	status ∈ dispatched received cancelled. Timestamps recorded per leg.
assignments	reference_no, base_id, equipment_type_id, quantity, personnel_name, personnel_id, personnel_rank, assigned_date, due_date, returned_quantity, expended_quantity, status, created_by	status ∈ active returned expended.
expenditures	assignment_id, base_id, equipment_type_id, quantity, reason, expended_on, remarks, created_by	assignment_id is NULL for a direct write-off. reason ∈ used-in-operation training spoilage lost damaged other.
stock_ledger	base_id, equipment_type_id, tx_type, direction, quantity, balance_after, ref_table, ref_id, occurred_at, created_by	Append-only. balance_after is stored for a readable running-balance history, but the authoritative closing balance is recomputed as the sum.
audit_logs	user_id, email, action, entity, entity_id, details, ip_address, user_agent, created_at	Written by middleware on every mutating and sensitive read request; queryable with filters from the UI.

3.3 Key integrity rules

- **Foreign keys are enforced** (`PRAGMA foreign_keys = ON`) with `ON DELETE CASCADE` for base-scoped rows and `ON DELETE RESTRICT` where history must not disappear (e.g. equipment types with ledger entries).
- **CHECK constraints** reject `quantity <= 0` , invalid statuses and invalid direction values at the database level, not only in application code.
- **Unique references** — `purchases.reference_no` and `transfers.reference_no` are unique, so a document number cannot be reused.
- **WAL journal mode** is enabled so reads are not blocked during writes.
- **Ledger identity verified by test** — the smoke suite asserts, for every base/equipment pair, that Opening + Purchases + Transfers In – Transfers Out – Assigned – Write-offs equals the closing balance, and that no balance is negative.

4. Authentication & Role-Based Access Control

4.1 Authentication flow

1. The user submits email and password to `POST /api/auth/login`.
2. The server fetches the user, verifies the bcrypt hash, and rejects the request if the account is inactive — with the same generic error message in both cases, so the endpoint does not reveal whether an email exists.
3. On success a JWT is signed with `JWT_SECRET`, containing the user id, role and base id, with an 8-hour expiry. The password hash is never included.
4. The client stores the token and re-attaches it as `Authorization: Bearer <token>` on every request via a single `api.js` wrapper.
5. On a 401 the client clears the session and redirects to the login screen.

4.2 Role permission matrix

Capability	Admin	Base Commander	Logistics Officer
View dashboard, balances, movements	All bases	Own base	Own base
View purchases / transfers / assignments	All bases	Own base	Own base
Create purchase	Yes	Yes	Yes
Dispatch / receive / cancel transfer	Yes	Yes	Yes
Create assignment / record expenditure	Yes	Yes	No — read only
User management (create / edit / delete)	Yes	No	No
View audit log	Yes	No	No

4.3 How enforcement works

Authorisation is enforced on the **server**; the frontend only hides controls the user cannot use, to reduce confusion. Two independent mechanisms are applied to every request:

- `requireRole(...)` — a route declares which roles may call it. A Logistics Officer calling `POST /api/assignments` receives `403 Forbidden`.
- `assertBaseAllowed()` — for non-admins, any supplied `base_id` is compared with the user's assigned base. Reading a record outside that base returns `403`; creating a record for another base is rejected. Queries for the dashboard and list endpoints are automatically filtered to the user's own base, so data leakage through a missing filter is not possible.

The Admin role also has `base_id = NULL`, which is the marker for "global access". A separate `GET /api/meta/permissions` endpoint returns the current user's capabilities so the UI can render the correct navigation and read-only banners.

Verified, not assumed

The test suite logs in as all three roles and asserts both the positive path (the role can do what it should) and the negative path (the role is refused what it must not). A test asserting only that a commander can create an assignment proves very little; the tests that prove the system is safe are the ones asserting a 403.

5. API Logging (Audit Trail)

5.1 What is logged

An `auditMiddleware` sits in the Express pipeline and records every request that changes state, plus a set of sensitive reads. Each entry captures who did what, to which entity, from where and when.

Column	Content
<code>user_id</code> , <code>email</code>	Who performed the action (email denormalised so the record survives user deletion)
<code>action</code>	Verb such as <code>login</code> , <code>create</code> , <code>update</code> , <code>delete</code> , <code>receive</code> , <code>return</code>
<code>entity</code> , <code>entity_id</code>	What was touched — <code>purchase</code> , <code>transfer</code> , <code>user</code> — and its identifier
<code>details</code>	Safe JSON summary of the change (see sanitiser below)
<code>ip_address</code> , <code>user_agent</code>	Origin of the request
<code>created_at</code>	UTC timestamp

5.2 Sensitive-field sanitiser

Audit details are built from request bodies, so a naive implementation would write passwords into the log. A recursive sanitiser strips any key matching a blocklist — `password` , `password_hash` , `token` , `secret` , `authorization` — replacing the value with "[redacted]" at any depth, including inside nested objects and arrays. Password change and user-create events are logged this way, so the audit trail can be shared with an auditor without leaking credentials.

5.3 Auditability of the stock itself

Beyond the request log, the domain is auditable by construction: purchases and transfers are never deleted, only counter-transacted; every stock change has a corresponding `stock_ledger` row pointing back at the source document (`ref_table` + `ref_id`); and every document records the user who created it. The Audit Log screen supports filtering by action, entity, date range and free text, and expands any entry to show its full detail.

6. Screens

All screenshots were captured automatically by `frontend/scripts/screenshots.js` against the running application.

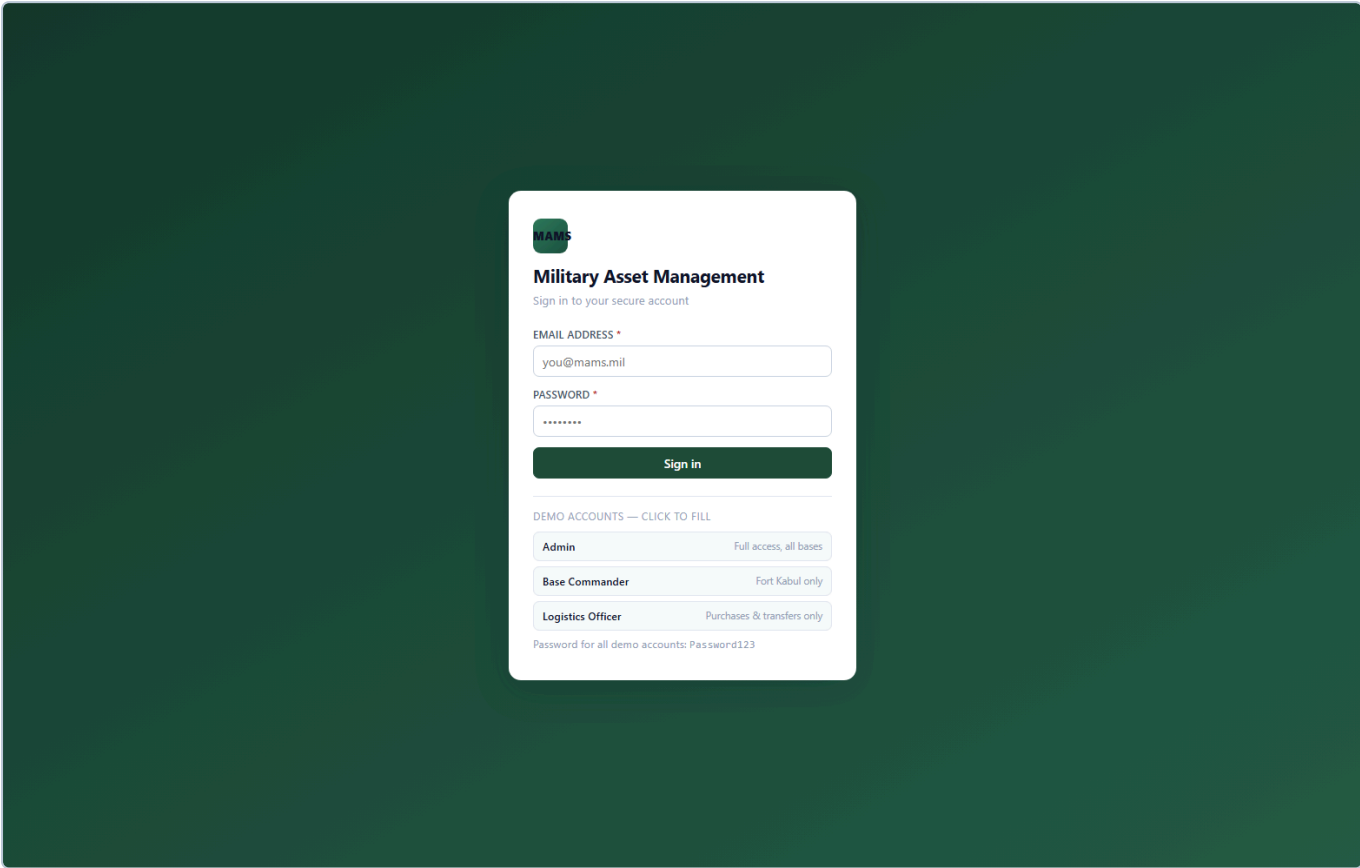


Figure 1 — Login screen with one-click demo credentials for each role.

MAMS

Asset Management

Military logistics & accountability

OPERATIONS

Dashboard

Purchases

Transfers

Assignments & Expenditures

ADMINISTRATION

User Management

API Audit Log

System Administrator

Admin · All Bases

Sign out

SA System Administrator Administrator

28 September 2026

Sign out

FROM

dd-mm-yyyy

TO

dd-mm-yyyy

QUICK RANGE

Choose...

BASE

All bases

EQUIPMENT TYPE

All equipment

Reset

How the figures are built: Closing = Opening + Purchases + Transfer In - Transfer Out - Assigned - Direct Write-offs

Net Movement = Purchases + Transfer In - Transfer Out · "Expended" is consumption booked against assignments, which happens off-base, so it is reported separately and is not part of the closing balance.

OPENING BALANCE

542,196

Stock held at period start

NET MOVEMENT

+126,043

+127,207 purchased · +38,209 in · -39,373 out

CLOSING BALANCE

661,158

Stock held at period end

ASSIGNED

7,076

Issued to personnel

EXPENDED

1,574

Consumed / written off

PURCHASES

127,207

Procured into bases

TRANSFER IN

38,209

Received from other bases

TRANSFER OUT

39,373

Sent to other bases

Movement by equipment type

Opening → closing reconciliation

EQUIPMENT	OPENING	PURCH.	T/IN	T/OUT	NET	CLOSING
120mm Mortar Shell Ammunition · per round	6,800	+1,507	+0	-1,200	+307	7,107
5.56x45mm Ammunition Ammunition · per round	385,000	+93,000	+20,000	-20,000	+93,000	473,500
7.62x51mm Ammunition Ammunition · per round	148,000	+32,000	+18,000	-18,000	+32,000	177,500
Body Armour Vest Equipment · per unit	760	+150	+140	-100	+190	910
Night Vision Goggles Equipment · per unit	250	+40	+0	-0	+40	287
Cargo Truck 5t Vehicle · per unit	60	+10	+3	-3	+10	69
HMMWV Armoured Vehicle · per unit	60	+10	+0	-4	+6	65
Toyota Hilux 4x4 Vehicle · per unit	144	+20	+6	-6	+20	159
.308 Sniper Rifle Weapon · per unit	46	+0	+0	-0	0	42
5.56mm Assault Rifle Weapon · per unit	780	+470	+60	-60	+470	1,229
7.62mm Machine Gun Weapon · per unit	102	+0	+0	-0	0	96
9mm Pistol Weapon · per unit	194	+0	+0	-0	0	194

Closing balance by base

BASE	OPENING	CLOSING
B-02 Camp Vance	163,852	205,108
B-03 Fob Khost	115,014	141,693
B-01 Fort Kabul	248,218	279,241
B-04 Naval Support Base	15,112	35,116

Recent movements

Ledger with running balance

DATE	TYPE	BASE	EQUIPMENT	QTY	BALANCE
28 Sept 2026	Purchase	Camp Vance	120mm Mortar Shell	+7	3,607
22 Sept 2026	Transfer In	Fort Kabul	Body Armour Vest	+40	350
22 Sept 2026	Transfer Out	Camp Vance	HMMWV Armoured	-4	26
20 Sept 2026	Write-Off	Fob Khost	Night Vision Goggles	-3	57
19 Sept 2026	Purchase	Fort Kabul	Night Vision Goggles	+40	150
18 Sept 2026	Transfer Out	Fort Kabul	120mm Mortar Shell	-1,200	2,000
15 Sept 2026	Transfer In	Camp Vance	Body Armour Vest	+100	320
14 Sept 2026	Write-Off	Camp Vance	Cargo Truck 5t	-1	8
13 Sept 2026	Assigned	Fob Khost	5.56x45mm Ammunition	-1,500	101,500
12 Sept 2026	Transfer Out	Fort Kabul	Body Armour Vest	-100	350
11 Sept 2026	Purchase	Fort Kabul	Cargo Truck 5t	+6	24
10 Sept 2026	Assigned	Fort Kabul	.308 Sniper Rifle	-4	18
09 Sept 2026	Assigned	Camp Vance	7.62x51mm Ammunition	-2,500	30,500
09 Sept 2026	Transfer In	Fob Khost	Cargo Truck 5t	+3	11
07 Sept 2026	Write-Off	Fort Kabul	HMMWV Armoured	-1	25
07 Sept 2026	Purchase	Fob Khost	5.56x45mm Ammunition	+18,000	103,000
06 Sept 2026	Transfer Out	Camp Vance	Cargo Truck 5t	-3	9
04 Sept 2026	Assigned	Fob Khost	5.56mm Assault Rifle	-9	351
04 Sept 2026	Transfer In	Fob Khost	7.62x51mm Ammunition	+10,000	38,000
03 Sept 2026	Assigned	Fort Kabul	Body Armour Vest	-40	310
03 Sept 2026	Purchase	Camp Vance	120mm Mortar Shell	+1,500	3,600
02 Sept 2026	Assigned	Camp Vance	Toyota Hilux 4x4	-3	51
01 Sept 2026	Transfer Out	Fort Kabul	7.62x51mm Ammunition	-10,000	74,000

31 Aug 2026	Assigned	Fort Kabul	5.56x45mm Ammunition	-3,000	202,000
29 Aug 2026	Assigned	Fort Kabul	Toyota Hilux 4x4	-2	68
29 Aug 2026	Transfer In	Fort Kabul	7.62x51mm Ammunition	+8,000	84,000
29 Aug 2026	Purchase	Fort Kabul	7.62x51mm Ammunition	+12,000	76,000
28 Aug 2026	Assigned	Camp Vance	7.62mm Machine Gun	-6	28
26 Aug 2026	Transfer Out	Camp Vance	7.62x51mm Ammunition	-8,000	33,000
25 Aug 2026	Assigned	Fort Kabul	5.56mm Assault Rifle	-12	468
23 Aug 2026	Transfer In	Camp Vance	Toyota Hilux 4x4	+6	54
21 Aug 2026	Transfer Out	Fort Kabul	Toyota Hilux 4x4	-6	70
19 Aug 2026	Purchase	Camp Vance	HMMWV Armoured	+10	30
15 Aug 2026	Purchase	Fort Kabul	5.56mm Assault Rifle	+200	540
14 Aug 2026	Transfer In	Fob Khost	5.56mm Assault Rifle	+60	360
12 Aug 2026	Purchase	Fob Khost	Toyota Hilux 4x4	+8	40
11 Aug 2026	Transfer Out	Fort Kabul	5.56mm Assault Rifle	-60	480
07 Aug 2026	Transfer In	Camp Vance	5.56x45mm Ammunition	+20,000	170,000
07 Aug 2026	Purchase	Camp Vance	5.56mm Assault Rifle	+150	410
04 Aug 2026	Transfer Out	Fort Kabul	5.56x45mm Ammunition	-20,000	205,000

Figure 2 — Dashboard: the four required headline metrics (Opening Balance, Net Movement, Assigned, Expended) plus Closing Balance, with breakdowns by equipment and by base.

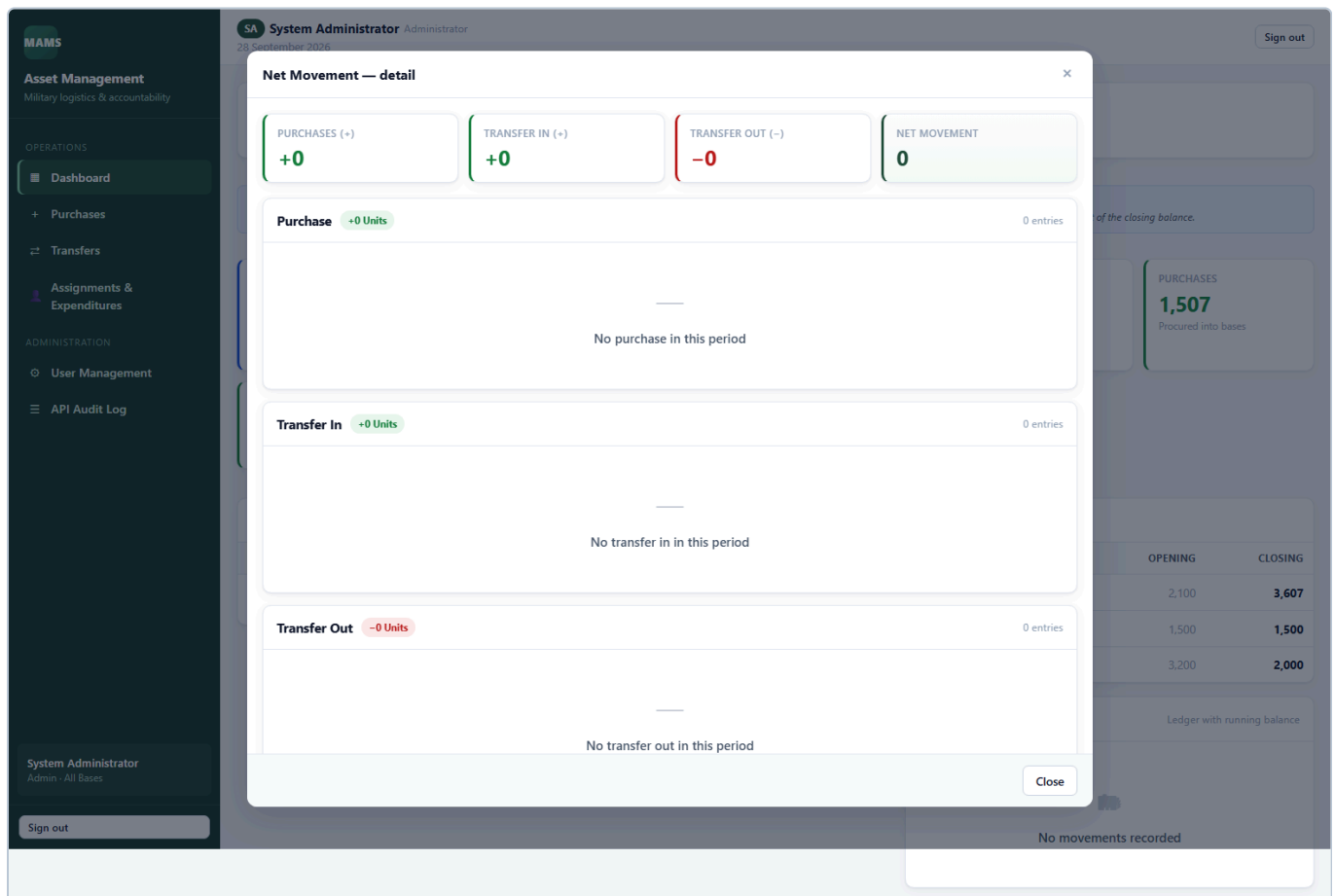


Figure 3 — The Net Movement pop-up (bonus feature): individual purchases, transfers in and transfers out behind the single headline figure.

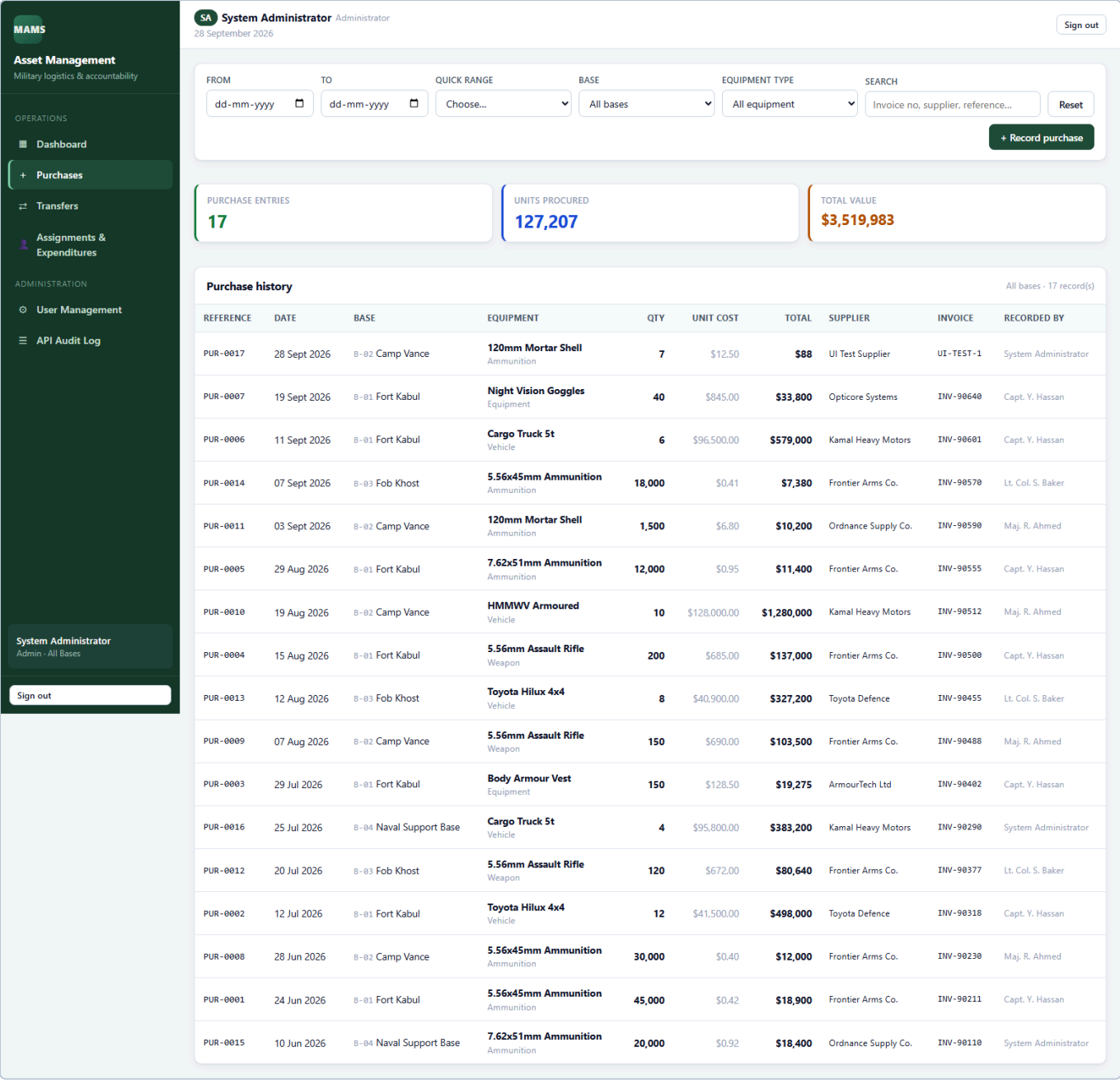


Figure 4 — Purchase history with date, base and equipment filters.

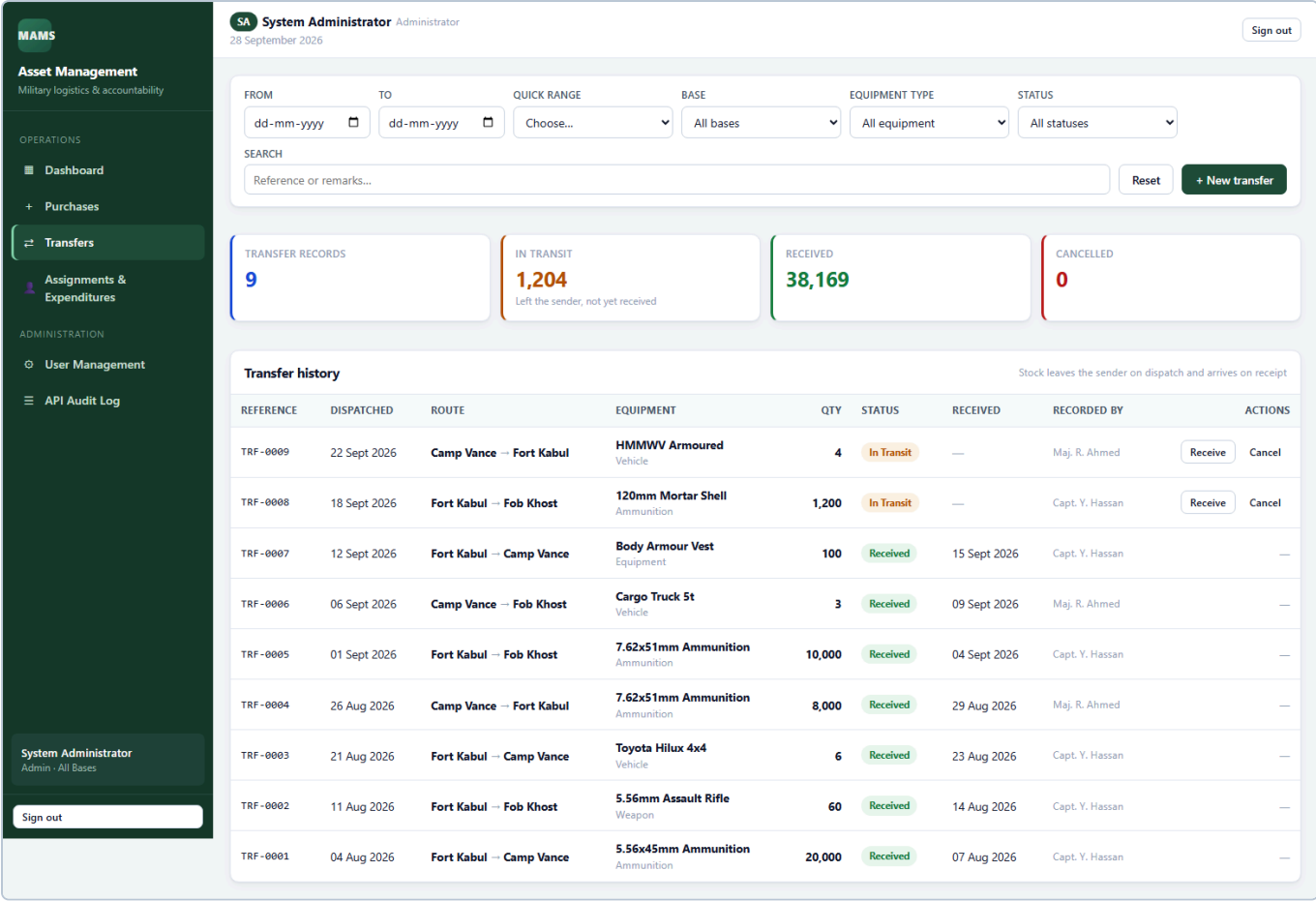


Figure 5 — Inter-base transfers with full status and timestamp history.

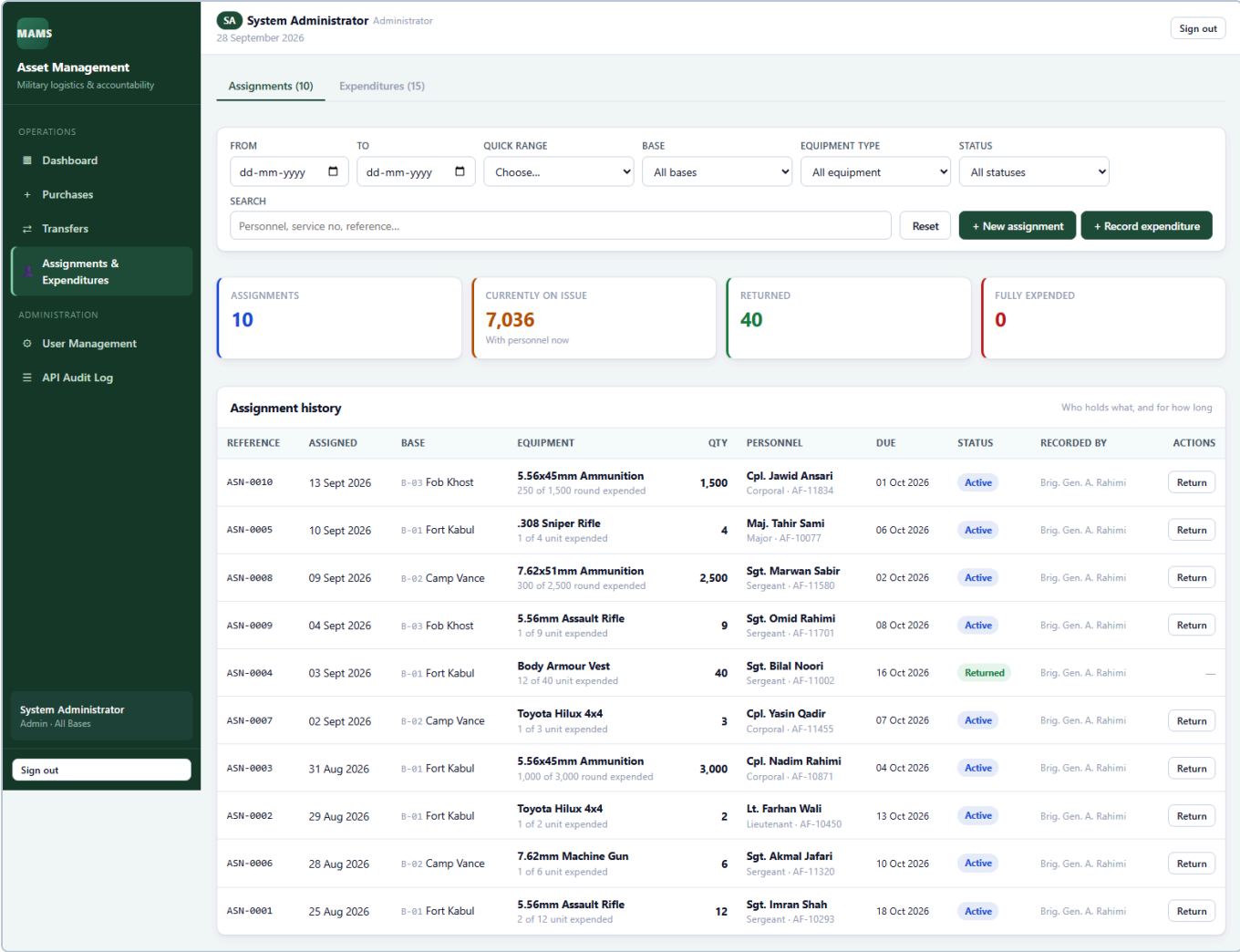


Figure 6 — Asset assignments to named personnel, with quantity assigned, returned and expended.

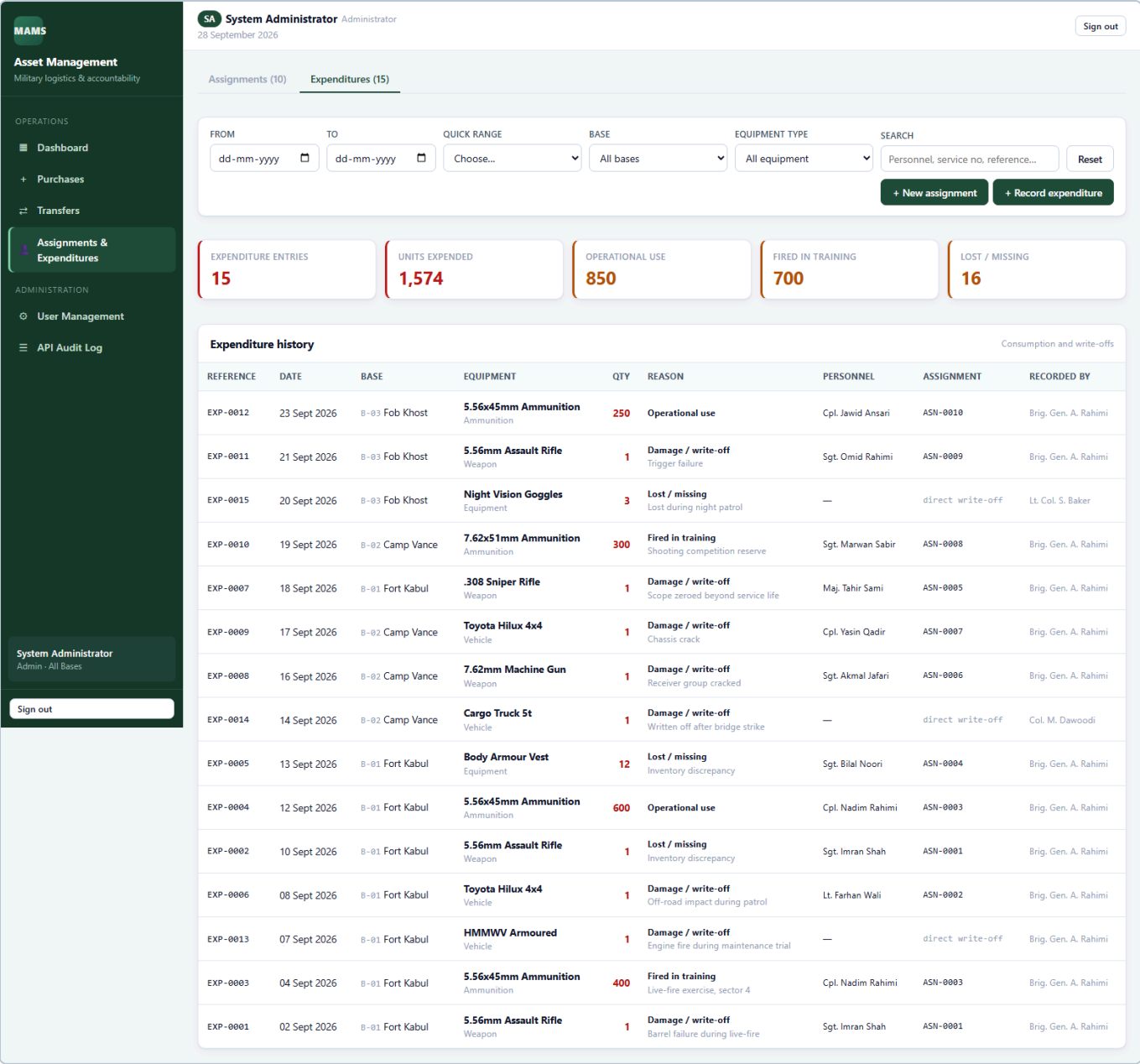


Figure 7 — Expenditure log with reason codes, filtered by assignment and base.

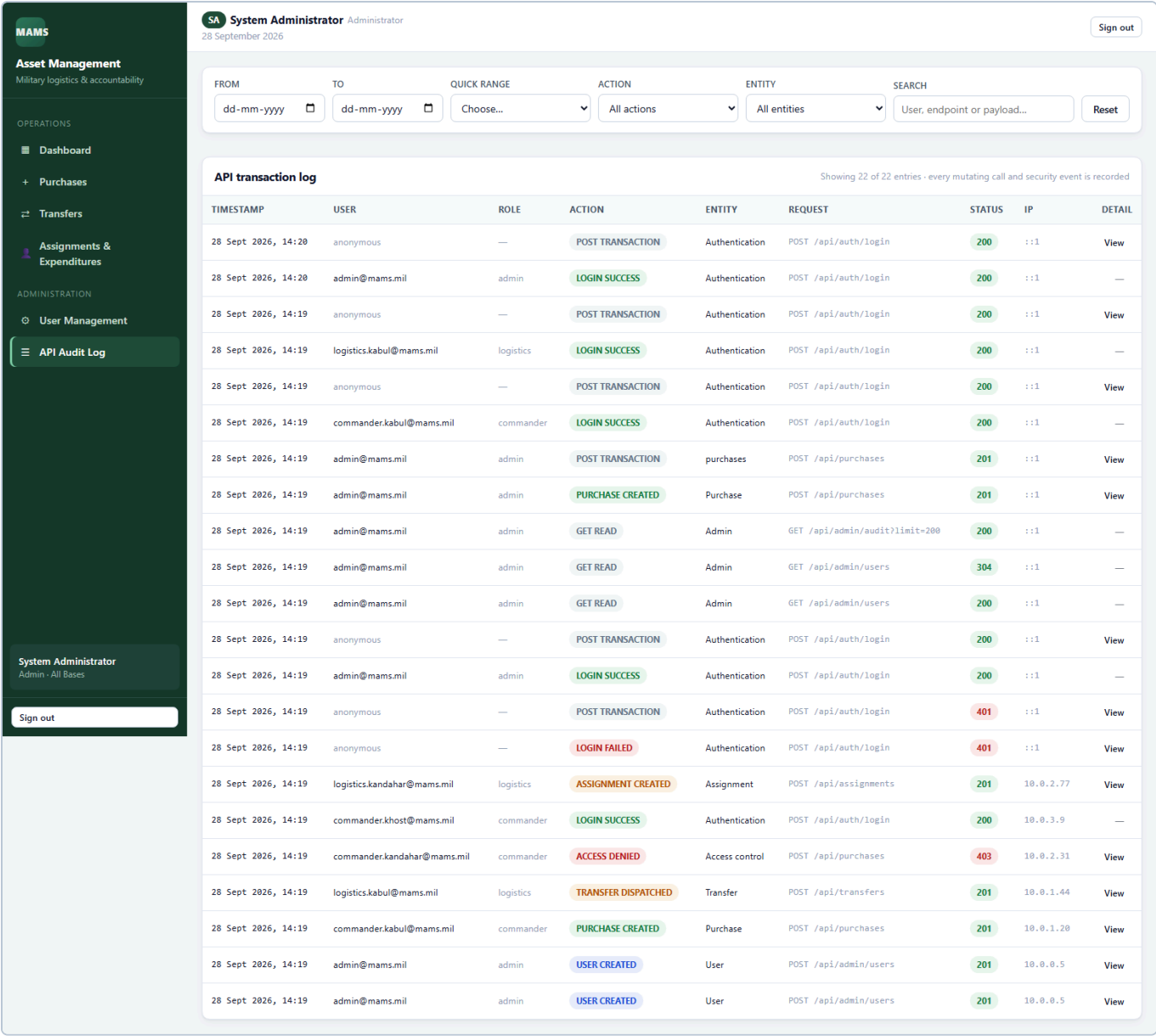


Figure 8 — API / activity audit log with filters and expandable detail rows.

MAMS

Asset Management

Military logistics & accountability

OPERATIONS

Dashboard

Purchases

Transfers

Assignments & Expenditures

ADMINISTRATION

User Management

API Audit Log

System Administrator

Admin - All Bases

Sign out

SA System Administrator Administrator

28 September 2026

Sign out

ACCESS CONTROL

Admins manage every base. Commanders and logistics officers are locked to the base assigned to their account.

+ Add user

Users

6 account(s)

NAME	EMAIL	ROLE	BASE	RANK	STATUS	CREATED	ACTIONS
System Administrator	admin@mams.mil	Admin	All bases	Colonel (Ret.)	Active	28 Sept 2026, 14:19	admin
Brig. Gen. A. Rahimi	commander.kabul@mams.mil	Commander	Fort Kabul	Brigadier General	Active	28 Sept 2026, 14:19	commander Disable Delete
Col. M. Dawoodi	commander.kandahar@mams.mil	Commander	Camp Vance	Colonel	Active	28 Sept 2026, 14:19	commander Disable Delete
Lt. Col. S. Baker	commander.khost@mams.mil	Commander	Fob Khost	Lieutenant Colonel	Active	28 Sept 2026, 14:19	commander Disable Delete
Capt. Y. Hassan	logistics.kabul@mams.mil	Logistics	Fort Kabul	Captain	Active	28 Sept 2026, 14:19	logistics Disable Delete
Maj. R. Ahmed	logistics.kandahar@mams.mil	Logistics	Camp Vance	Major	Active	28 Sept 2026, 14:19	logistics Disable Delete

Figure 9 — User management (admin only): create users and assign role and base.

**YH****Capt. Y. Hassan** logistics · Fort Kabul

28 September 2026

Scoped To Fort Kabul**Sign out**

FROM

dd-mm-yyyy



TO

dd-mm-yyyy



QUICK RANGE

Choose...



EQUIPMENT TYPE

All equipment

**Reset**

How the figures are built: Closing = Opening + Purchases + Transfer In - Transfer Out - Assigned - Direct Write-offs

Net Movement = Purchases + Transfer In - Transfer Out ·

"Expended" is consumption booked against assignments, which happens off-base, so it is reported separately and is not part of the closing balance.

OPENING BALANCE

248,218

Stock held at period start

NET
MOVEMENT**CLICK FOR
DETAIL ▸****+34,082**

+57,408 purchased · +8,040 in
· -31,366 out

CLOSING BALANCE

279,241

Stock held at period end

ASSIGNED

3,058

Issued to personnel

EXPENDED

1,017

Consumed / written off

PURCHASES

57,408

Procured into bases

TRANSFER IN

8,040

Received from other bases

TRANSFER OUT

31,366

Sent to other bases

Movement by equipment type

Opening → closing reconciliation

EQUIPMENT

OPENING

PURCH.

T/IN

T/OUT

120mm

Mortar Shell

Ammunition ·
per round

3,200

+0

+0

-1,200

5.56x45mm

Ammunition

180 000

+45 000

+0

-20 000

Ammunition · per round	100,000	+40,000	+0	-20,000
---------------------------	---------	---------	----	---------

7.62x51mm

Ammunition Ammunition · per round	64,000	+12,000	+8,000	-10,000
--	--------	---------	--------	---------

Body

Armour Vest Equipment · per unit	300	+150	+40	-100
---	-----	------	-----	------

Night Vision

Goggles Equipment · per unit	110	+40	+0	-0
---	-----	-----	----	----

Cargo Truck

5t Vehicle · per unit	18	+6	+0	-0
------------------------------------	----	----	----	----

HMMWV

Armoured Vehicle · per unit	26	+0	+0	-0
--	----	----	----	----

Toyota

Hilux 4x4 Vehicle · per	64	+12	+0	-6
-----------------------------------	----	-----	----	----

unit

.308 Sniper Rifle

Weapon · per unit

22 +0 +0 -0

5.56mm Assault Rifle

Weapon · per unit

340 +200 +0 -60

7.62mm Machine Gun

Weapon · per unit

48 +0 +0 -0

9mm Pistol

Weapon · per unit

90 +0 +0 -0

Closing balance by base

BASE

OPENING

CLOSING

B-01 Fort Kabul

248,218

279,241

Recent movements

Ledger with running balance



No movements recorded

Figure 10 — Responsive layout: the same dashboard on a phone-width screen.

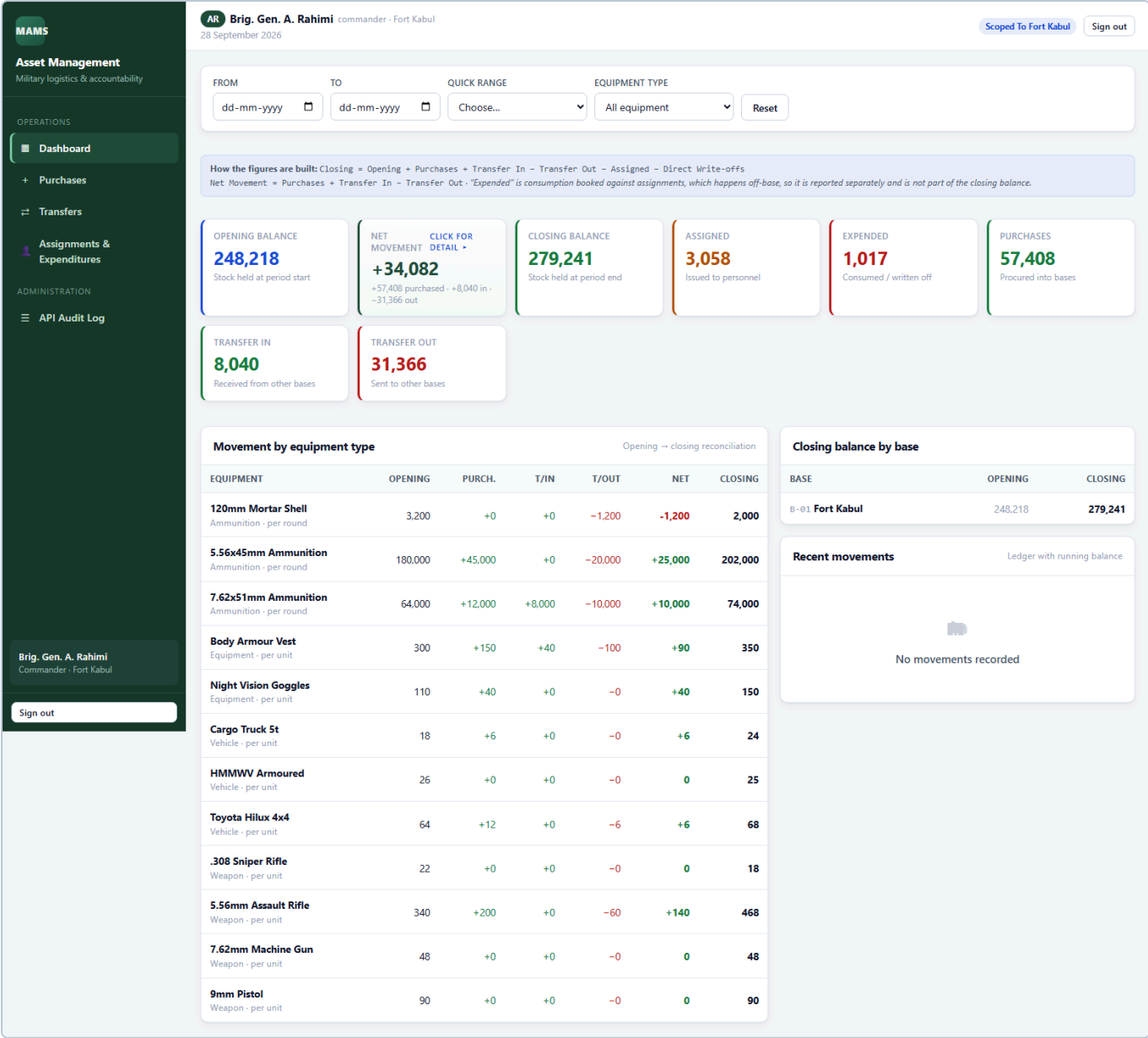


Figure 11 — The same dashboard as a Base Commander: automatically scoped to his own base only.

7. Installation & Running the Project

7.1 Prerequisites

- **Node.js 20 or newer** (developed and tested on Node 22 / 25) — check with `node -v`
- **npm 10 or newer** — ships with Node
- No database server, no Docker, no Python required.

7.2 Start the backend

```
cd backend
npm install
cp .env.example .env      # Windows: copy .env.example .env
npm start
```

The database file `backend/database/military_assets.db` is created automatically and populated with demo data on first run, so there is no separate migration or import step. Verify it with `http://localhost:4000/api/health`, which should return `{"status":"ok"}`.

7.3 Start the frontend

```
cd frontend
npm install
cp .env.example .env      # Windows: copy .env.example .env
npm run dev
```

Open `http://localhost:5173`. Vite proxies `/api` to port 4000, so the browser talks to a single origin and CORS never enters the picture during development.

7.4 Environment variables

Variable	Default	Description
<code>PORT</code>	<code>4000</code>	Port the API listens on
<code>DB_FILE</code>	<code>database/military_assets.db</code>	Path to the SQLite file, relative to <code>backend/</code>
<code>JWT_SECRET</code>	<code>dev fallback</code>	Must be set to a long random value in production
<code>JWT_EXPIRES_IN</code>	<code>8h</code>	Token lifetime
<code>CORS_ORIGIN</code>	<code>localhost:5173</code>	Comma-separated allowed origins in production
<code>VITE_API_URL</code>	<code>empty (use proxy)</code>	Frontend only: absolute API base URL when deployed separately

7.5 Re-seeding, dumping and testing

```
cd backend
npm run reset      # wipe and re-seed the demo database
npm run dump       # write deliverables/database/dump.sql
npm run smoke      # re-seed, then run 77 API + RBAC assertions

cd ../frontend
npm run build       # production bundle in dist/
node scripts/browser-check.js # 39 real-browser assertions
node scripts/screenshots.js  # regenerate all screenshots
```

7.6 Restoring the supplied database dump

```
sqlite3 military_assets.db < dump.sql
# or simply copy deliverables/database/military_assets.db into backend/database/
```

8. API Endpoints

All endpoints are prefixed with `/api` and speak JSON. Every route except `POST /auth/login` and `GET /health` requires the header `Authorization: Bearer <token>`.

8.1 Authentication & reference data

Method	Endpoint	Access	Description
GET	<code>/health</code>	public	Liveness probe used by the host platform.
POST	<code>/auth/login</code>	public	Body <code>{email, password}</code> . Returns <code>{token, user}</code> . Audited.
GET	<code>/auth/me</code>	any	Current user profile from the token.
GET	<code>/meta/bases</code>	any	List of bases — narrowed to the user's own base for non-admins.
GET	<code>/meta/equipment-types</code>	any	Equipment catalogue.
GET	<code>/meta/permissions</code>	any	Capabilities of the current role, used to render the UI.

8.2 Dashboard

Method	Endpoint	Description
GET	<code>/dashboard/summary</code>	All KPI totals for the selected window, with per-equipment and per-base breakdowns. Accepts <code>from</code> , <code>to</code> , <code>baseId</code> , <code>equipmentTypeId</code> .
GET	<code>/dashboard/net-movement</code>	Pop-up drill-down. The individual purchases, transfer-ins and transfer-outs that make up the Net Movement figure.
GET	<code>/dashboard/balances</code>	Current stock for every base × equipment combination.
GET	<code>/dashboard/movements</code>	Ledger rows for the window with the running <code>balance_after</code> ; accepts <code>limit</code> .

8.3 Purchases

Method	Endpoint	Access	Description
GET	<code>/purchases</code>	any	History filtered by <code>from</code> , <code>to</code> , <code>baseId</code> , <code>equipmentTypeId</code> , free-text <code>q</code> .
GET	<code>/purchases/summary</code>	any	Document count, total units and total value for the same filters.
POST	<code>/purchases</code>	admin, commander, logistics	Records a purchase and posts a <code>PURCHASE</code> ledger entry. Rejects a base the user does not control.

8.4 Transfers

Method	Endpoint	Access	Description
GET	/transfers	any	History where the user's base is either end of the route; <code>status</code> filter available.
POST	/transfers	admin, commander, logistics	Dispatch. Validates sufficient stock at the sender, then posts <code>TRANSFER_OUT</code> .
POST	/transfers/:id/receive	admin, commander, logistics	Confirms arrival at the destination and posts <code>TRANSFER_IN</code> there.
POST	/transfers/:id/cancel	admin, commander, logistics	Cancels an in-transit transfer, returning the stock to the sender.

8.5 Assignments & expenditures

Method	Endpoint	Access	Description
GET	/assignments	any	History with <code>status</code> filter; shows assigned, returned and expended quantities.
POST	/assignments	admin, commander	Issues assets to named personnel and posts <code>ASSIGNMENT</code> ledger entries.
POST	/assignments/:id/return	admin, commander	Returns the un-expended remainder to base stock.
GET	/expenditures	any	History with a <code>reason</code> filter.
POST	/expenditures	admin, commander	Records consumption against an assignment, or a direct write-off with a reason and remarks.

8.6 Administration & audit

Method	Endpoint	Access	Description
GET	/admin/users	admin	List users. Password hashes are explicitly excluded from the query.
POST	/admin/users	admin	Create a user with a role and base.
PATCH	/admin/users/:id	admin	Change role, base assignment or active flag.
DELETE	/admin/users/:id	admin	Delete a user.
GET	/admin/audit	admin	Audit log filtered by <code>action</code> , <code>entity</code> , <code>from</code> , <code>to</code> , <code>q</code> .

8.7 Error handling

A single error handler converts anything thrown into a consistent JSON body, `{ "error": "message" }`, with the correct status: `400` for validation problems, `401` for missing or invalid tokens, `403` for role or base violations, `404` for unknown records and routes, and `409` for conflicts such as an insufficient stock balance or a duplicate reference number.

9. Login Credentials

All demo accounts share the password `Password123`. They are seeded by `npm run seed` and are listed on the login screen for one-click access.

Email	Password	Role	Base / scope
<code>admin@mams.mil</code>	<code>Password123</code>	Admin	All bases — full access
<code>commander.kabul@mams.mil</code>	<code>Password123</code>	Base Commander	Fort Kabul (B-01)
<code>commander.kandahar@mams.mil</code>	<code>Password123</code>	Base Commander	Camp Vance (B-02)
<code>commander.khost@mams.mil</code>	<code>Password123</code>	Base Commander	Fob Khost (B-03)
<code>logistics.kabul@mams.mil</code>	<code>Password123</code>	Logistics Officer	Fort Kabul — purchases & transfers
<code>logistics.kandahar@mams.mil</code>	<code>Password123</code>	Logistics Officer	Camp Vance — purchases & transfers

Demo credentials only

These accounts exist so the system can be evaluated immediately. Before any real use, remove or re-password the seeded accounts, set a strong `JWT_SECRET`, and serve over HTTPS.

10. Testing & Verification

Suite	Checks	Coverage
API smoke suite <code>backend/scripts/smoke.js</code>	77 / 77	Login and token handling, role and base scoping, the ledger identity and zero-negative-balance invariants, purchase/transfer/assignment/expenditure lifecycles, insufficient-stock rejection, admin user management, audit entries, and error responses.
Browser suite <code>frontend/scripts/browser-check.js</code>	39 / 39	Real Chrome via the DevTools Protocol: login and rejection of bad credentials, every page renders, forms open and validate, filters change the results, the Net Movement pop-up opens and closes, the admin-only page is hidden for other roles, mobile layout, and no console errors or failed network requests.

Both suites exit non-zero on failure and print a pass/fail line, so they can be wired into CI directly. The verification is behavioural rather than cosmetic: the tests assert that a Logistics Officer is refused an assignment, that the dashboard figures equal the sum of the transaction pages, and that cancelling a transfer restores the sender's balance.

11. Deployment

A `render.yaml` blueprint is included for the API, and `vercel.json` and `netlify.toml` for the frontend, so the application can be pushed to a live URL without modification. The frontend is a static build; the backend reads its port, database path and allowed origins from the environment.

```
Backend → Render      : rootDir=backend, buildCommand=npm install, startCommand=npm start,
                        healthCheckPath=/api/health
Frontend → Vercel      : rootDir=frontend, buildCommand=npm run build, outputDirectory=dist
Frontend → Netlify     : base directory=frontend, build=npm run build, publish=dist
                        environment: VITE_API_URL=https://<api-host>
```

Read this before demonstrating on a free host

SQLite is a single file. On a host with an ephemeral disk the database is reset and re-seeded on each cold start, so any records created during a demo will not persist. This does not affect the application itself — attach a persistent disk, or run locally with `npm run reset` between demonstrations.

12. Conclusion

The delivered system meets the specified requirements: it reports opening balance, closing balance and net movement; distinguishes purchases, transfers in and transfers out; tracks assigned and expended assets with named personnel; and enforces role-based access control that is verified by tests rather than assumed. Two additions go beyond the brief — the Net Movement pop-up that opens the headline figure down to individual transactions, and a responsive layout that makes the same application usable on a phone in the field.

The design choice that makes the reporting trustworthy is the single append-only ledger: because the dashboard is a projection of the transaction history rather than a separately maintained counter, the figures cannot drift apart from the records behind them. The main constraints on further work are the local-file database and the stateless token, both of which have a clear, contained upgrade path described in the limitations section.